Design Pattern

Prepared by: Team 03

Instructor
Md Samsuddoha

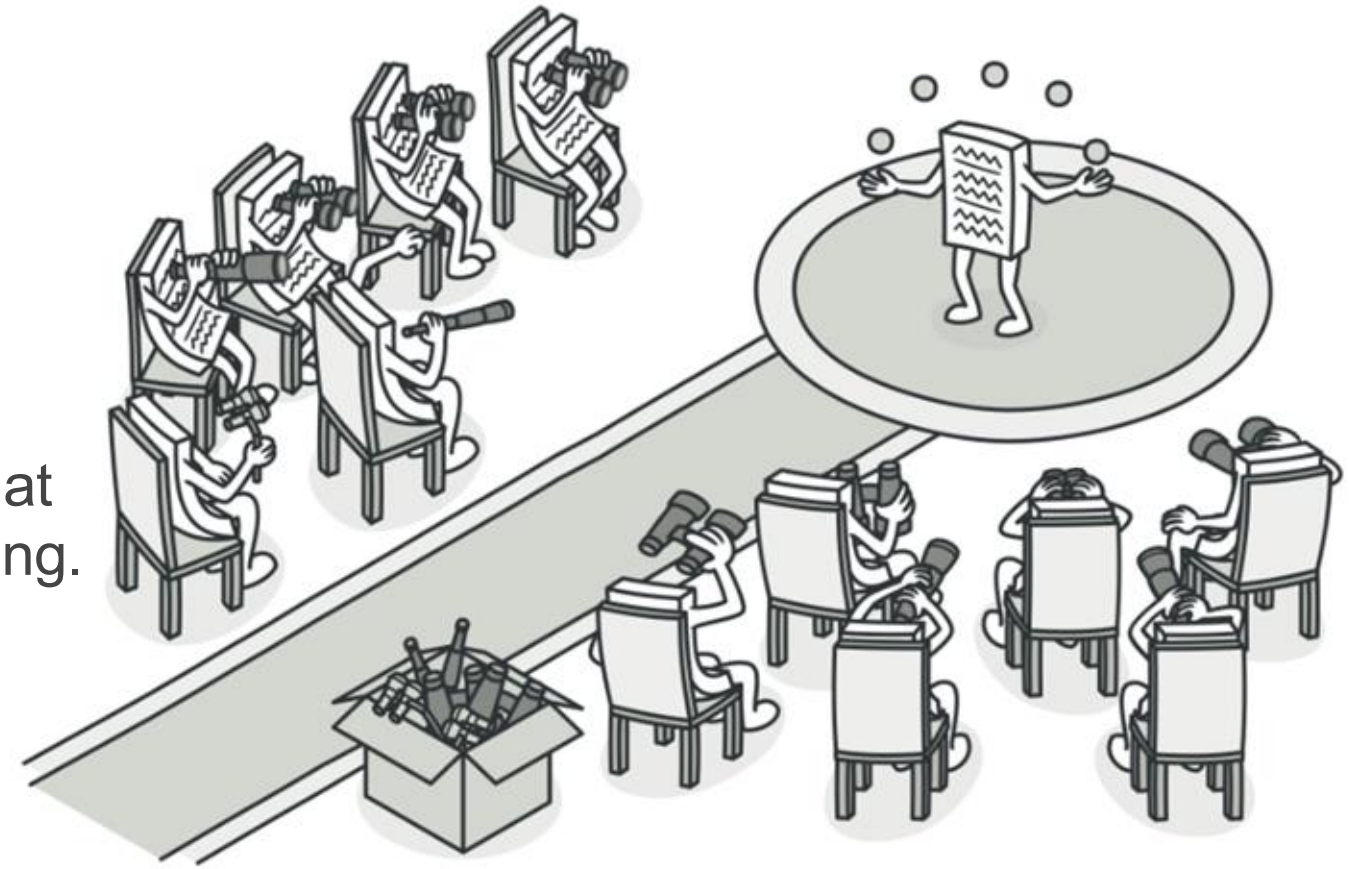
Assistant Professor
Department of Computer Science & Engineering, University of Barishal

The Observer Pattern

An essential behavioral design pattern that establishes a dynamic, one-to-many subscription model to notify multiple dependent objects of state changes automatically.

WHAT IS THE OBSERVER PATTERN?

Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they’re observing.

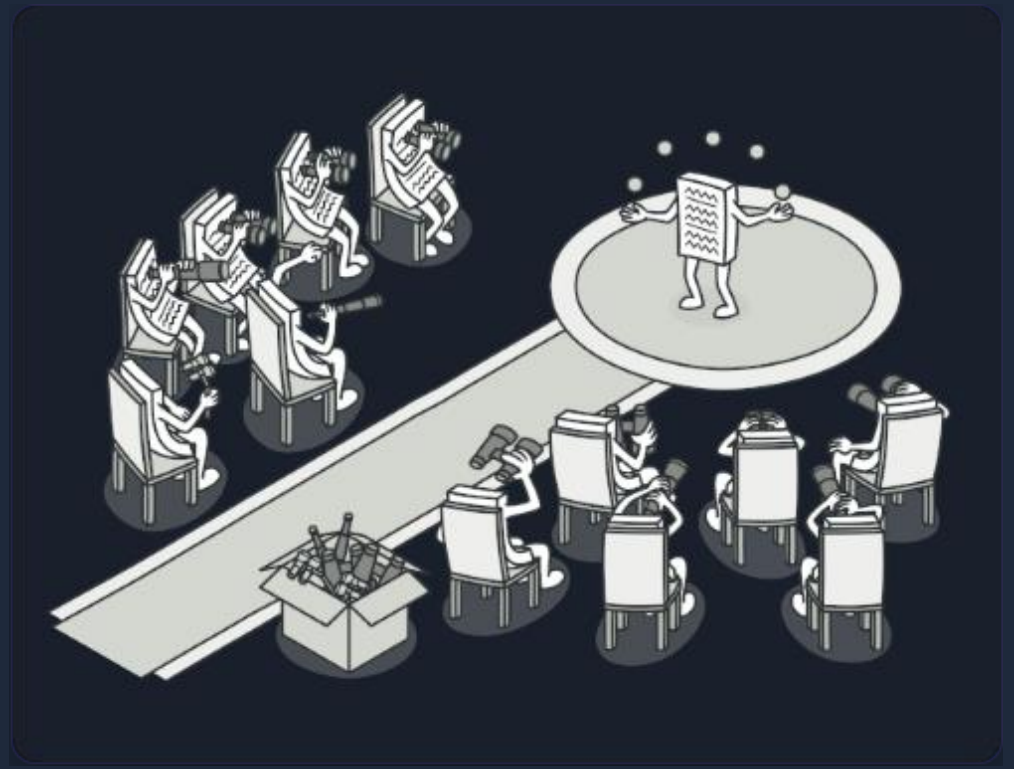


INTENT: BEHAVIORAL BROADCAST SYSTEMS

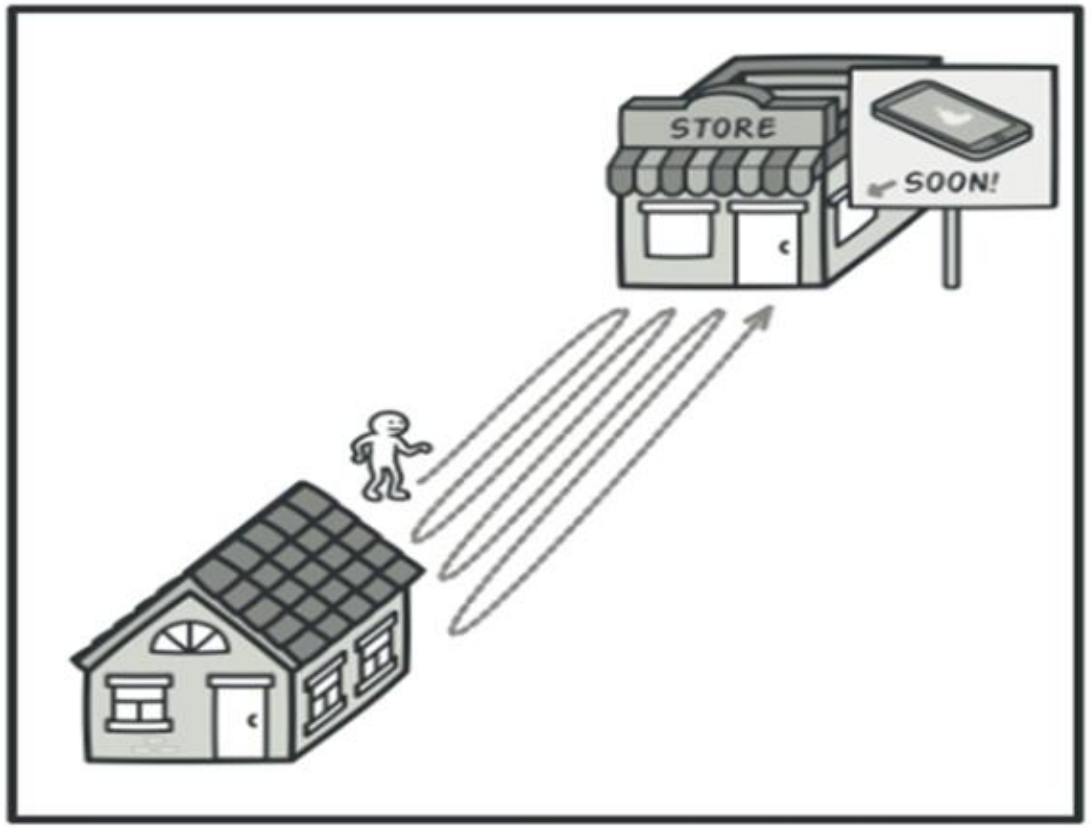
The **Observer** pattern defines a one-to-many dependency between objects, ensuring that when one object changes state, all of its dependents are notified and updated automatically.

Standard Subscription Model

This design allows components to dynamically monitor a target object's state changes without tight coupling.



Problem I: Polling



The Wasteful Check

Imagine a customer waiting for a new phone release. They could visit the store every hour (Polling).

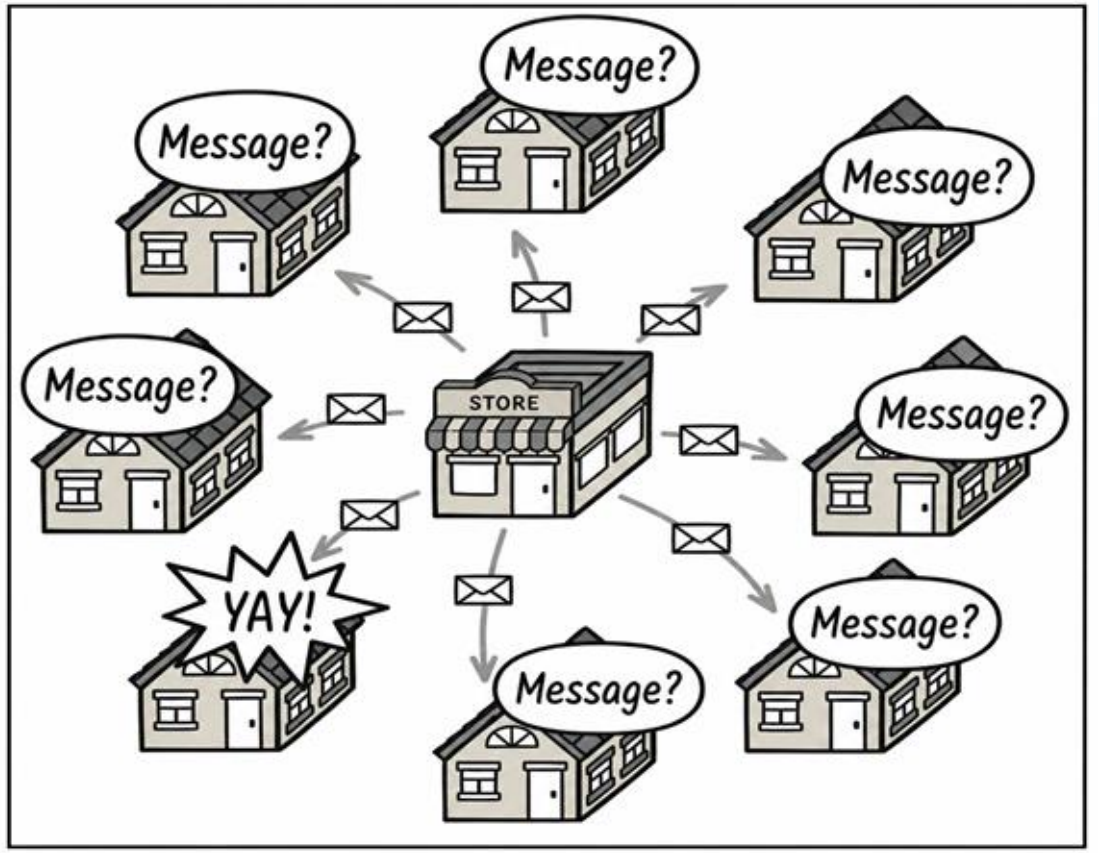
Most trips are wasted. In code, constant checking for state changes consumes CPU cycles and battery life unnecessarily.

Problem II: Spam

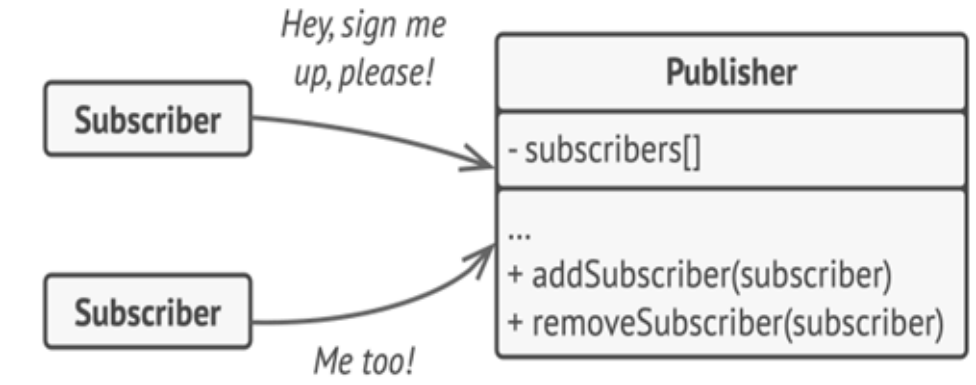
The Broadcast Trap

Alternatively, the store could send an email to EVERY customer in their database for every new product. This is "Spam".

Objects that don't care about the event are still forced to process the notification, causing performance bottlenecks.



Solution I: Subscription



A subscription mechanism lets individual objects subscribe to event notifications.

The Registry Mechanism

The "Publisher" maintains a list of objects interested in its events. It provides methods to join or leave this list.

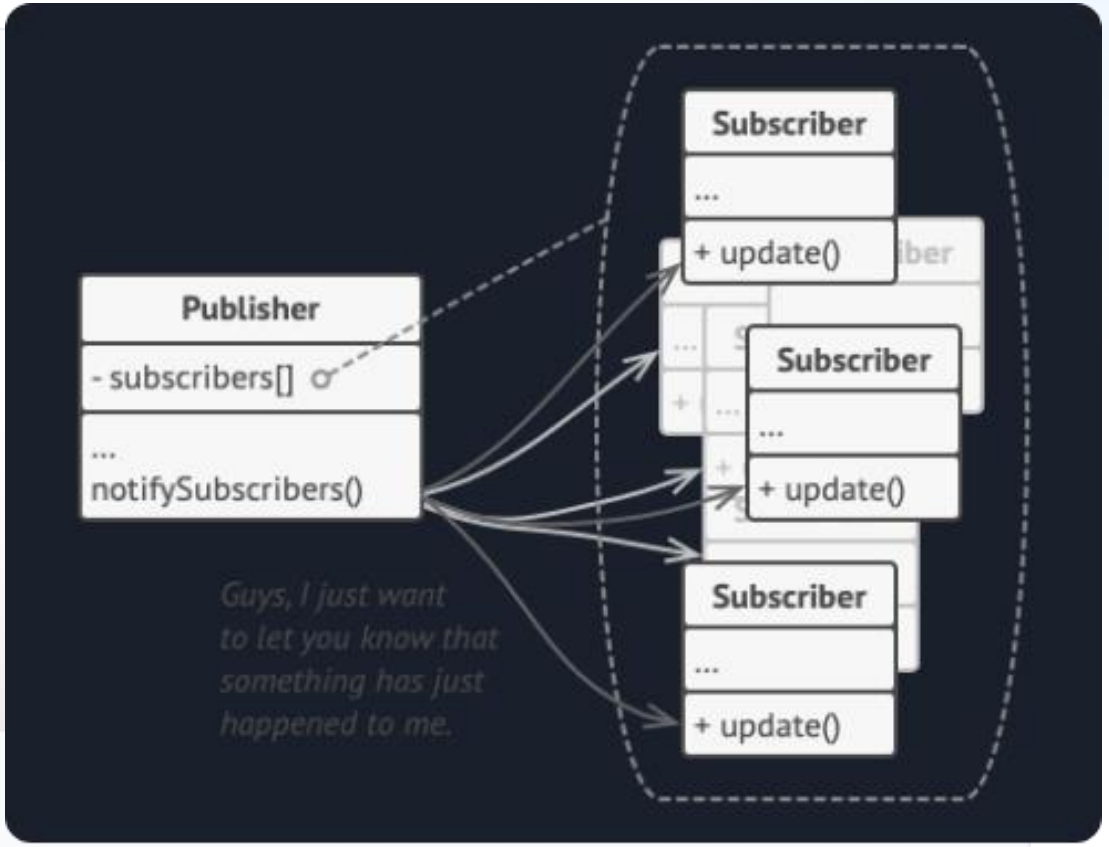
When an event occurs, it iterates through the registry and calls an update method on each subscriber.

Solution II: Pushing Updates

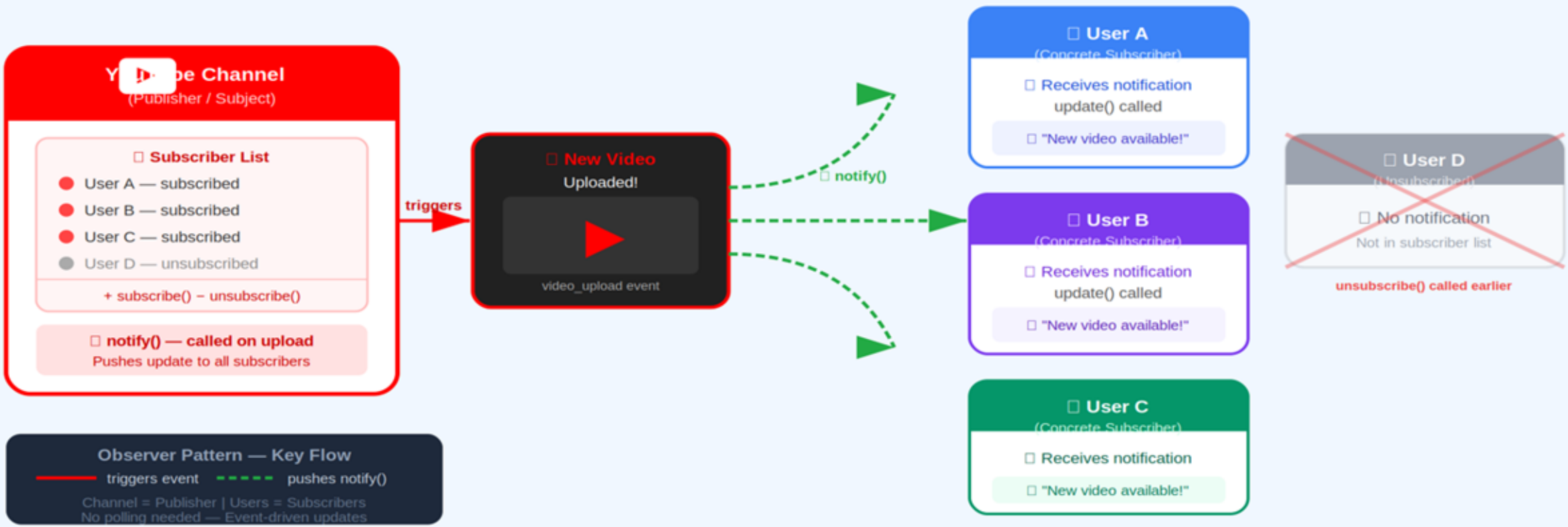
The "Push" Model

Instead of the client asking "is it ready?", the publisher pushes the information only to those who signed up.

This ensures **efficiency** (no wasted polls) and **relevancy** (no spam for non-subscribers).

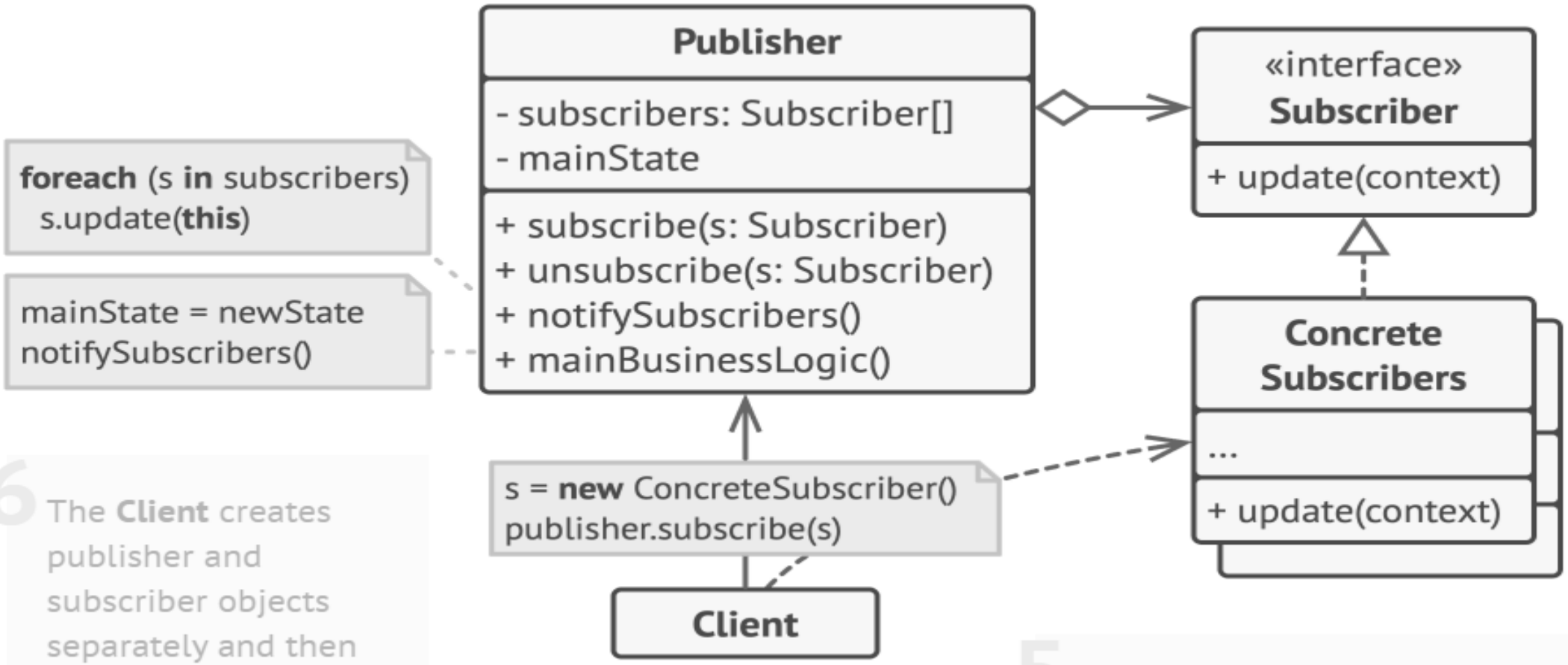


REAL-WORLD ANALOGY: YOUTUBE CHANNEL NOTIFICATIONS



Subscribers do not need to repeatedly check for new videos. The YouTube channel maintains a list of subscribers and automatically notifies only those users whenever a new video is uploaded. Users can unsubscribe at any time.

UML Architecture



Concrete Subscribers react based on the Publisher's broadcast.

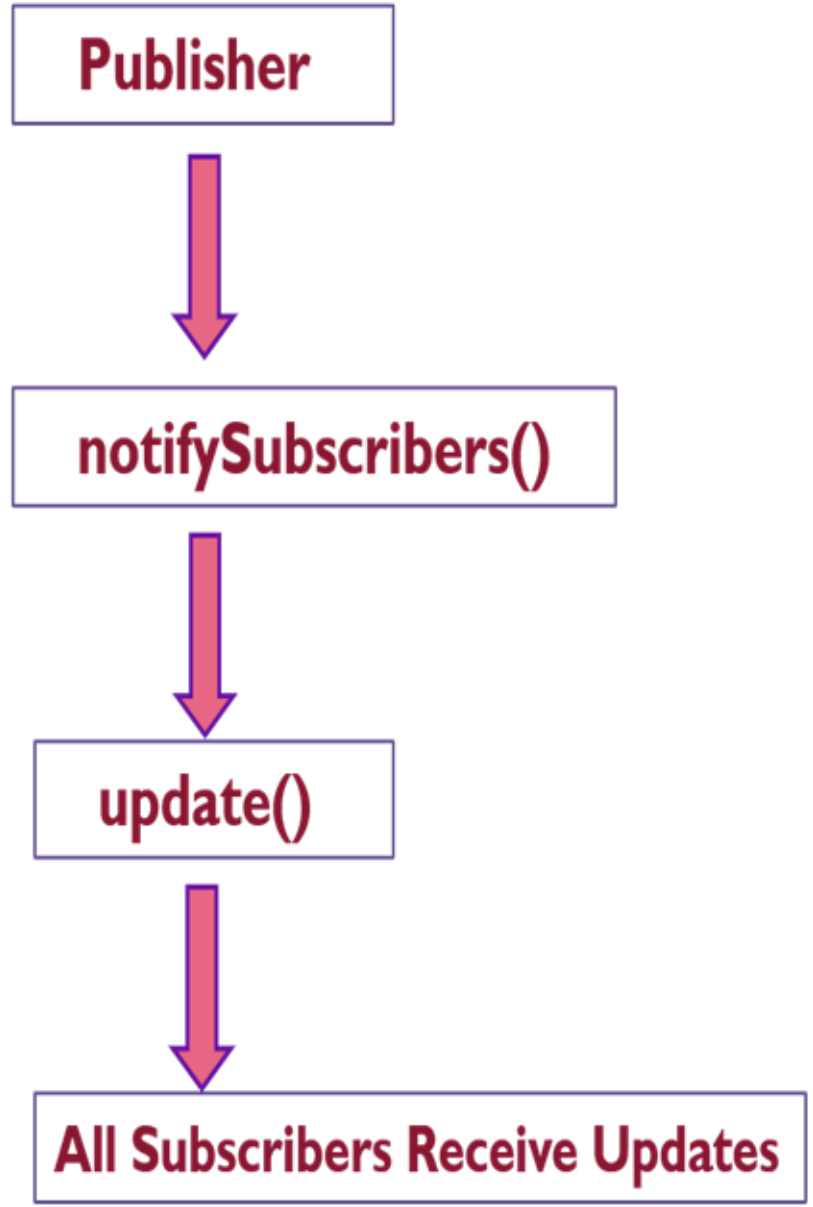
Pseudocode For Observer Pattern

```
BEGIN

INTERFACE Subscriber
        METHOD update()


CLASS ConcreteSubscriber IMPLEMENTS Subscriber

        METHOD update()
                DISPLAY "Subscriber received update"
```

```
CLASS Publisher

        LIST subscribers

        METHOD subscribe(subscriber)
                ADD subscriber TO
                subscribers

        METHOD unsubscribe(subscriber)
                REMOVE subscriber FROM
                subscribers

        METHOD notifySubscribers()

                FOR EACH subscriber IN
                subscribers
                        subscriber.update()
```

```
MAIN PROGRAM

        CREATE publisher

        CREATE subscriber1
        CREATE subscriber2

        publisher.subscribe(subscriber1)
        publisher.subscribe(subscriber2)

        publisher.notifySubscribers()

END
```

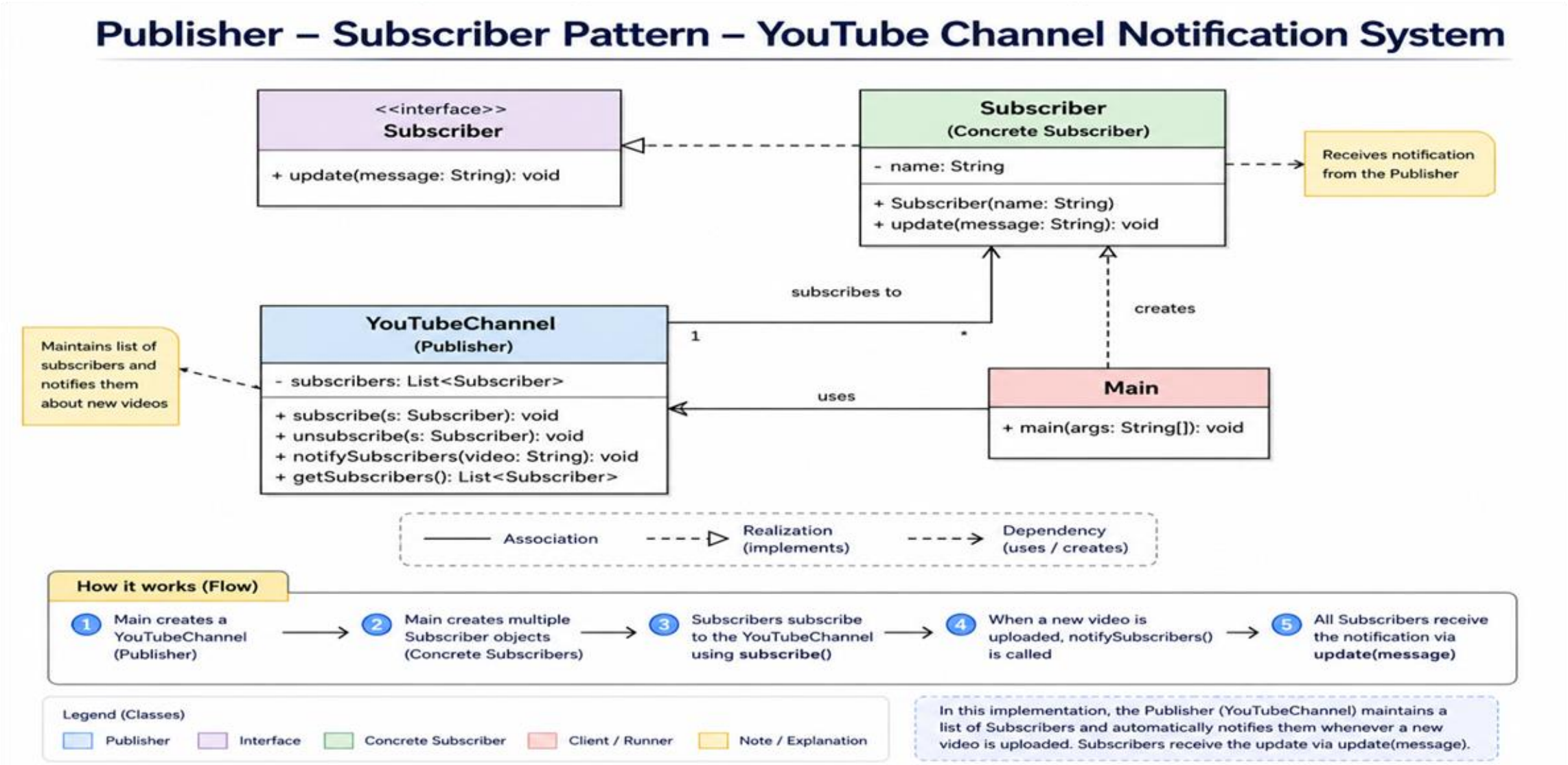
Note :The object that has some interesting state is often called *subject*, but since it's also going to notify other objects about the changes to its state, we'll call it *publisher*. All other objects that want to track changes to the publisher's state are called *subscribers*.



One publisher sends updates, and all connected subscribers automatically receive them.

Implementation of Observer Pattern (using JAVA)

In this implementation, the Observer Pattern is demonstrated using a YouTube channel notification system, where subscribers automatically receive updates whenever a new video is uploaded.



Observer.java

```
// ____________________________Observer Interface
public interface Observer {
    void update(String message);
}
```

YouTubeChannel.java

```
import java.util.*;
public class YouTubeChannel {

    private List<Observer> subscribers = new ArrayList<>();
    // ------Add observer
    public void subscribe(Observer observer) {
        subscribers.add(observer);
    }
    // __________Notify all observers
    public void notifySubscribers(String videoTitle) {

        for (Observer observer : subscribers) {
            observer.update(videoTitle);
        }
    }
}
```

Subscriber.java

```
// Concrete Observer
public class Subscriber implements Observer {

    private String name;

    public Subscriber(String name) {
        this.name = name;
    }

@Override
    public void update(String message) {
        System.out.println(name + " received notification: " +
message);
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
     // ---Create Subject
        YouTubeChannel channel = new YouTubeChannel();
        // -_______Create Observers
        Subscriber user1 = new Subscriber("ALice");
        Subscriber user2 = new Subscriber("Bob");
        Subscriber user3 = new Subscriber("Polycarp ");
        Subscriber user4 = new Subscriber("Monocarp");
        // _____________Subscribe observers
        channel.subscribe(user1);
        channel.subscribe(user2);
        channel.subscribe(user3);
        channel.subscribe(user4);
        // _________________Send notification
        channel.notifySubscribers("New Java Tutorial Uploaded!");
    }
}
```

Ouput

```
[Running] cd "f:\Pattern Design\observerPattern\" && javac Main.java && java Main
ALice received notification: New Java Tutorial Uploaded!
Bob received notification: New Java Tutorial Uploaded!
Polycarp  received notification: New Java Tutorial Uploaded!
Monocarp received notification: New Java Tutorial Uploaded!
```

Applicability

GUI Events

Hooking custom code to buttons, menus, and text fields.

Data Sync

Updating multiple views (tables, charts) when database changes.

Temporal

Monitoring objects for a limited time or specific cases.

HOW TO IMPLEMENT STEP-BY-STEP

1

Separate Logic

Divide your business logic into core publisher features and optional subscriber actions.

2

Standard Contract

Declare the Subscriber interface, defining a generic update method.

3

Registry Methods

Add attachment methods to the publisher to let subscribers join or leave dynamically.

4

Broadcast Cycle

Trigger the notification loop within the publisher whenever its internal state changes.

The Pros: Why use it?

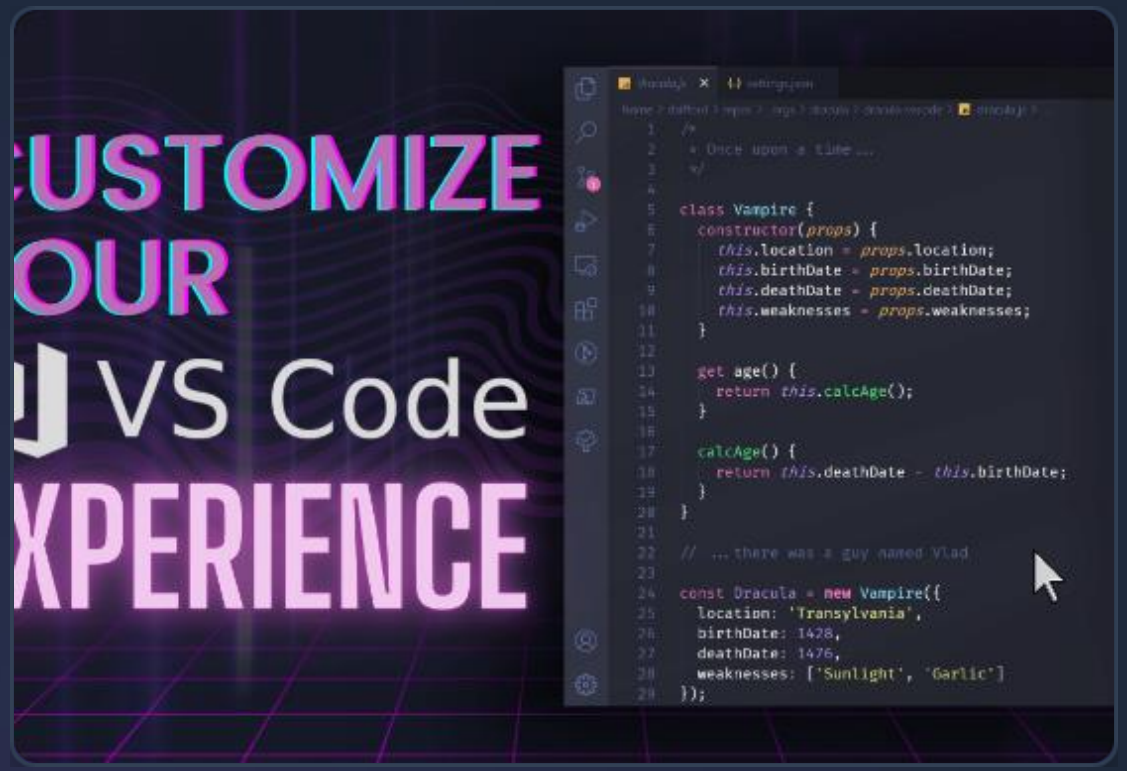
Open/Closed

You can introduce new subscriber classes without having to change the publisher's code.

Runtime Relations

You can establish relations between objects on the fly, subscribing or unsubscribing as needed.

The Cons: The Trade-offs



Random Order

Subscribers are notified in random order. You cannot guarantee which object will be updated first.

Overhead: Unnecessary updates if subscribers aren't carefully managed.

Leak Danger: Forgotten unsubscriptions can result in memory leakage (lapsed listeners).

RELATIONS WITH A PATTERNS

Chain of Responsibility, Command, Mediator, and Observer address various ways of connecting senders and receivers of requests:

Chain of Resp.

Passes a request sequentially along a dynamic chain of potential receivers until one of them handles it.

Command

Establishes unidirectional connections between senders and receivers.

Mediator

Eliminates direct connections between components, forcing them to communicate via a central mediator.

Observer

Lets receivers dynamically subscribe to and unsubscribe from receiving requests at runtime.

MEDIATOR VS. OBSERVER

Mediator Core Goal

The primary goal of Mediator is to eliminate mutual dependencies among a set of system components.

- Centralizes complex communications inside a single mediator instance.
- All components depend on one coordinator instead of referencing each other.

Observer Core Goal

The primary goal of Observer is to establish dynamic, one-way connections between objects.

- Allows some objects to act as subordinates of others.
- Establishes dynamically configured push pipelines to propagate events.

Pattern Synergy

Better Together

In complex GUI frameworks, **Abstract Factory** is used to create the specific buttons and windows (Mac vs Win), while **Observer** is used to handle the button clicks and updates. Construct with AF; Communicate with Observer.

QUESTIONS?

Thank you for your attention!